Sistemas Distribuidos

Taller: Agregador resiliente de APIs

Clientes de dependencias externas

`app/clients` y `tests/clients`

Autor (esta parte): Emerson

Rol: consumo resiliente de dependencias

Paquete	`app/clients/`
Pruebas	`tests/clients/`
API externa elegida	USGS (terremotos)
Servicio inestable	`servicio_inestable.py` (puerto 9000)
Estado	20 pruebas en verde

21 de octubre de 2026

Índice

1. Introduccion y contexto **2**
1.1. Que se evalua (rubrica) . . . 2

2. Arquitectura general **2**

3. Mi alcance y el contrato **3**
3.1. Decision de diseno: API por clases . . . 3

4. Jerarquia de excepciones (`__init__.py`) **3**

5. Cliente resiliente (`unstable_service.py`) **4**
5.1. C2 - Timeout explicito . . . 5
5.2. C3 - Reintentos selectivos . . . 5
5.3. C4 - Backoff exponencial con *full jitter* . . . 6
5.4. C5 - `Retry-After` y bloqueo compartido por IP . . . 6
5.5. C6 - Limite de concurrencia . . . 6
5.6. C7 - Paginacion con reintento por pagina . . . 6
5.7. Degradacion parcial y logging . . . 7

6. Cliente de la API externa (`external_api.py`) **7**
6.1. Validacion Pydantic y el hallazgo de `extra="forbid"` . . . 7
6.2. Cache TTL y resultado enriquecido . . . 7
6.3. C8 - Medicion de latencia (media y p95) . . . 8

7. Pruebas automatizadas (`tests/clients/`) **8**

8. Evidencias reales **9**
8.1. Reintentos, `Retry-After` y degradacion parcial . . . 9
8.2. Latencia con y sin cache (C8) . . . 10

9. Hallazgos para el equipo (analisis) **10**

10. Conclusiones **11**

A. Guion sugerido para la exposicion **11**

B. Comandos de reproduccion **11**

1. Introduccion y contexto

El taller pide construir una **API agregadora resiliente**: un servicio propio (FastAPI) que *agrega* dos dependencias que no controlamos:

1. un **servicio local inestable** (sensores) que falla de forma aleatoria (503), responde lentamente, corta conexiones y aplica limites de tasa (429 con `Retry-After`);
2. una **API publica real** por Internet (en este caso, los terremotos de **USGS**).

Mi responsabilidad es unicamente la capa de **clientes**: la pieza que sabe *hablar* con esas dos dependencias y *sobrevivir* cuando se portan mal. El resto del proyecto lo cubren mis companeras:

- **Nataly**: `app/config.py`, `app/models.py`, `app/cache.py`.
- **Daniela**: `app/routers/*`.
- **Grupal**: `main.py`, `README.md`, `requirements.txt`.

1.1. Que se evalua (rubrica)

Criterio	Puntos	Donde cae
Consumo de dependencias	15	`app/clients/`
API propia (endpoints)	20	routers (Daniela)
Resiliencia	30	`app/clients/`
Cache + latencia	10	`app/clients/`
Idempotencia	10	routers (Daniela)
Analisis / hallazgos	15	informe (grupo)
Bonus	+10	extras bien justificados

Cuadro 1: Reparto de la rubrica y de quien lo cubre.

2. Arquitectura general

La figura 1 ubica mi parte dentro del sistema completo. Las flechas discontinuas son dependencias que *yo no controlo*: por eso el cliente debe ser defensivo.

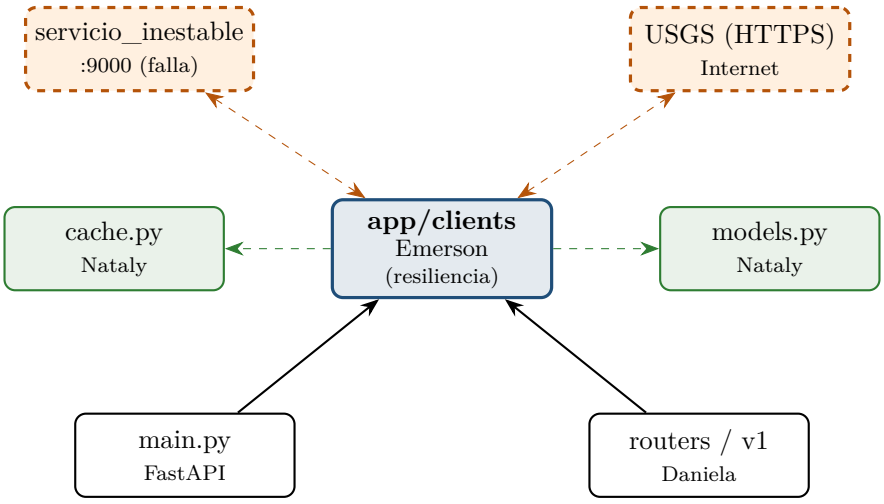


Figura 1: Arquitectura general. En azul, mi parte; en naranja punteado, las dependencias que no controlo; en verde, los modulos de Nataly.

3. Mi alcance y el contrato

Solo toco estos archivos (nada de mis companeras):

Archivo	Responsabilidad
`__init__.py`	Jerarquia de excepciones del contrato. Se define aqui (y no en los submodulos) para evitar importaciones circulares.
`unstable_service.py`	Cliente resiliente del servicio de sensores: timeouts, reintentos selectivos, backoff, `Retry-After`, bloqueo por IP, paginacion, degradacion parcial.
`external_api.py`	Cliente de USGS: dos llamadas encadenadas (feed → detalle), validacion Pydantic, cache con TTL y medicion de latencia.
`tests/clients/`	Pruebas automatizadas (pytest + respx) y evidencia real.

Cuadro 2: Archivos de mi parte (dentro de `app/clients/`).

3.1. Decision de diseno: API por clases

El esqueleto original define los clientes como **clases** y `main.py` / el router de Daniela ya las instancian asi (`ExternalEarthquakeClient`, `UnstableServiceClient`). Por eso **mantengo las clases** y conservo sus firmas, y ademas expongo funciones equivalentes del contrato (`get_con_reintentos`, `obtener_ultimo_sismo`). Cambiar a un estilo solo-funciones habria roto el codigo de mis companeras.

4. Jerarquia de excepciones (`__init__.py`)

El router traduce las excepciones a codigos HTTP. Para que eso funcione sin tocar su codigo, las excepciones nuevas **heredan** de las que ya se usaban (`UnstableServiceError` y `ExternalApiError`) y ademas de `TimeoutError` cuando corresponde (para que el 504 se dispare solo).

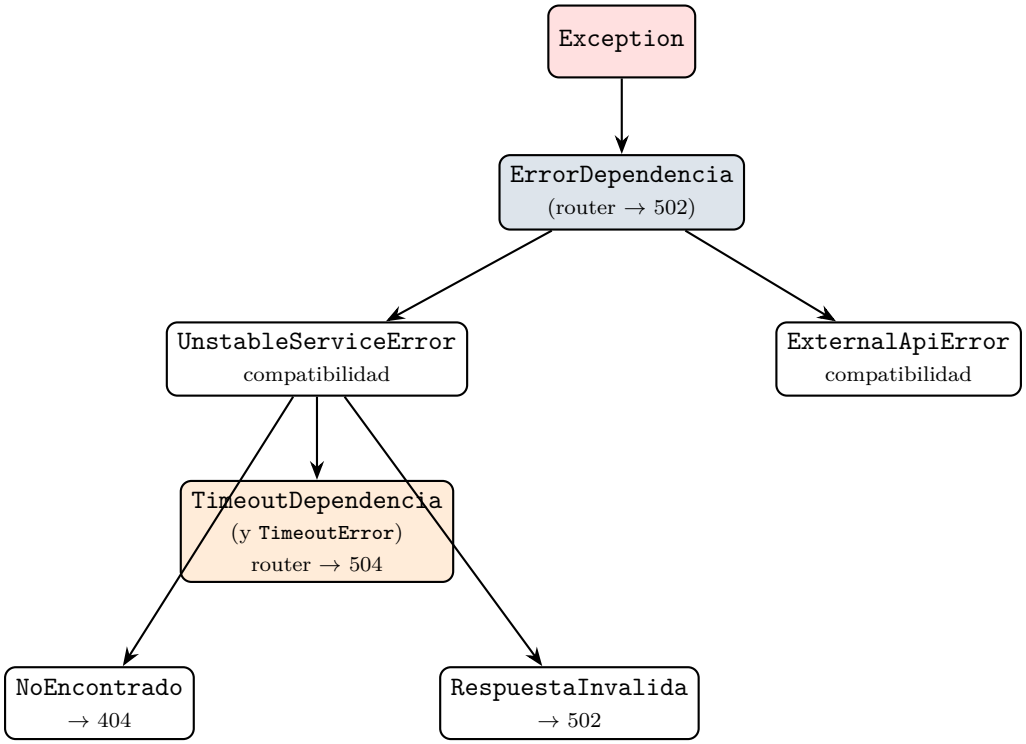


Figura 2: Jerarquia de excepciones. Las nuevas heredan de las de compatibilidad, asi el router sigue funcionando.

Listing 1: app/clients/___init___.py (fragmento)

```python
class ErrorDependencia(Exception):
    """Fallo de una dependencia que no se puede resolver (el router -> 502)."""

class UnstableServiceError(ErrorDependencia):
    """Error del servicio inestable. Compatibilidad con main.py/router (502)."""

class ExternalApiError(ErrorDependencia):
    """Error de la API externa. Compatibilidad con main.py/router (502)."""

class TimeoutDependencia(UnstableServiceError, TimeoutError):
    """La dependencia no respondio en el tiempo permitido (router -> 504)."""

class NoEncontrado(UnstableServiceError):
    """El recurso no existe en la dependencia (404 de origen -> 404)."""

class RespuestaInvalida(UnstableServiceError):
    """La respuesta no cumple el modelo Pydantic esperado (router -> 502)."""
```

Idea clave para explicar: `TimeoutDependencia` hereda de `TimeoutError`; el router atrapa `asyncio.TimeoutError` para el 504, asi que un timeout real del cliente cae *naturalmente* en el 504.

5. Cliente resiliente (`unstable_service.py`)

Este archivo concentra el **30 % de la rubrica**. Cada mecanismo es un requisito independiente.

Req.	Mecanismo	Evidencia en codigo
C2	Timeout explicito (conexion/lectura separadas)	`_tiempo_timeout`
C3	Reintentos selectivos (solo 503/429/timeout)	`get_con_reintentos`
C4	Backoff exponencial + *full jitter*	`_espera_backoff`
C5	`Retry-After` + bloqueo compartido por IP	`_marcar_bloqueo`, `_no_antes_de`
C6	Limite de concurrencia	`_semaforo = Semaphore(3)`
C7	Paginacion que sigue `siguiente`	`get_all_sensors`
—	Degradacion parcial	`obtener_lecturas`
—	Logging de cada intento	`_registrar`

Cuadro 3: Requisitos cubiertos por el cliente resiliente.

5.1. C2 - Timeout explicito

La lectura se acota a **2 s** deliberadamente *por debajo* de la lentitud simulada (4 s): asi una respuesta lenta cuenta como *timeout*, que si es reintentable.

Listing 2: Timeout separado conexion/lectura

```python
TIMEOUT_CONEXION = 2.0
TIMEOUT_LECTURA = 2.0   # menor que los 4 s de lentitud simulada

def _tiempo_timeout(timeout_seconds: float) -> httpx.Timeout:
    return httpx.Timeout(
        connect=TIMEOUT_CONEXION,
        read=min(timeout_seconds, TIMEOUT_LECTURA),
        write=timeout_seconds,
        pool=timeout_seconds,
    )
```

5.2. C3 - Reintentos selectivos

Solo se reintentan **503**, **429** y **timeouts**. Los errores legitimas de cliente (400, 404) se devuelven tal cual para que la capa superior los traduzca (404 → 404).

Listing 3: Bucle de reintentos selectivos

```python
for intento in range(1, max_intentos + 1):
    await _esperar_turno()           # respeta el bloqueo compartido tras 429
    try:
        async with _semaforo:        # maximo 3 peticiones simultaneas
            respuesta = await client.get(url, params=params, timeout=timeout)
    except httpx.TimeoutException as exc:
        if intento >= max_intentos:
            raise TimeoutDependencia(...) from exc
        espera, origen = _espera_backoff(intento, backoff_base)
        _registrar(intento, max_intentos, "timeout", espera, origen)
        await _dormir(espera)
        continue

    if respuesta.status_code == 429: ...
    if respuesta.status_code == 503: ...

    return respuesta   # 400, 404 y demas: NO se reintentan
```

5.3. C4 - Backoff exponencial con *full jitter*

La espera es aleatoria entre 0 y el tope exponencial. El jitter evita que muchas tareas reintenten *a la vez* (efecto manada).

Listing 4: Calculo de la espera (full jitter)

```python
def _espera_backoff(intento: int, base: float) -> tuple[float, str]:
    tope = min(BACKOFF_TOPE, base * (2 ** (intento - 1)))
    return random.uniform(0.0, tope), "backoff"
```

5.4. C5 - `Retry-After` y bloqueo compartido por IP

Si el servicio responde 429 con `Retry-After`, esa espera **manda** sobre el backoff. Ademas se marca un instante global `_no_antes_de`: *todas* las tareas lo respetan (el servicio bloquea por IP, no por peticion).

Listing 5: Bloqueo global por IP

```python
_lock_espera = asyncio.Lock()
_no_antes_de = 0.0

async def _marcar_bloqueo(segundos: float) -> None:
    global _no_antes_de
    async with _lock_espera:
        _no_antes_de = max(_no_antes_de, time.monotonic() + segundos)

async def _esperar_turno() -> None:
    async with _lock_espera:
        restante = _no_antes_de - time.monotonic()
    if restante > 0:
        await _dormir(restante)
```

5.5. C6 - Limite de concurrencia

Un semaforo evita saturar al servicio (y provocar mas 429).

Listing 6: Semaforo de concurrencia

```python
_semaforo = asyncio.Semaphore(3)   # maximo 3 peticiones a la vez
```

5.6. C7 - Paginacion con reintento por pagina

Se sigue `siguiente` hasta `null`. El reintento ocurre *dentro de cada pagina*: que falle una no obliga a repetir todo.

Listing 7: Paginacion completa

```python
async def get_all_sensors(self) -> list[Sensor]:
    sensores: list[Sensor] = []
    cursor: str | None = None
    while True:
        pagina = await self._pedir_pagina(cursor)   # reintenta por pagina
        sensores.extend(
            Sensor(id=crudo.id, ubicacion=crudo.ubicacion)
            for crudo in pagina.sensores
        )
        if pagina.siguiente is None:
            return sensores
        cursor = pagina.siguiente
```

5.7. Degradacion parcial y logging

`obtener_lecturas` pide varios sensores en paralelo y **no deja que uno malo tumbe al resto**: devuelve la dupla (lecturas ok, fallidos con motivo). Cada intento se registra con `logging` (causa, espera y origen), lo que produce la evidencia de reintentos.

Listing 8: Degradacion parcial

```
resultados = await asyncio.gather(
    *(self.obtener_lectura(sid) for sid in sensor_ids),
    return_exceptions=True,
)
for sensor_id, resultado in zip(sensor_ids, resultados):
    if isinstance(resultado, Lectura):
        lecturas.append(resultado)
    else:
        fallidos.append({"sensor_id": sensor_id, "motivo": str(resultado)})
```

6. Cliente de la API externa (`external_api.py`)

USGS se consume con **dos llamadas encadenadas**:

1. el *feed* `all_hour` (`UsgsFeed`) para hallar el evento mas reciente;
2. el *detalle* de ese evento (`UsgsFeature`).

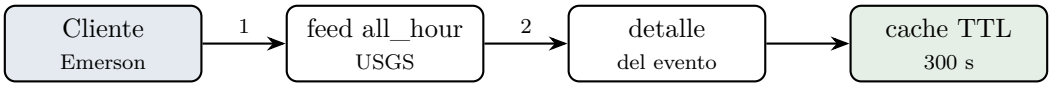


Figura 3: Consulta encadenada feed → detalle, con cache TTL.

6.1. Validacion Pydantic y el hallazgo de `extra="forbid"`

Los modelos de `models.py` (de Nataly) declaran `extra="forbid"`. Pero el feed **real** de USGS trae campos adicionales (`bbox`, `tz`, `sig`, `magType`, `felt`...). Resultado: contra la API real, la validacion fallaba y el endpoint habria devuelto **502**.

Solucion dentro de mi alcance (sin tocar `models.py`): antes de validar, se *recortan* las claves desconocidas usando los propios campos del modelo. Sigo validando con los modelos de Nataly, pero tolero los extras.

Listing 9: Tolerancia a campos extra de USGS

```
def _recortar(modelo: type[BaseModel], datos: dict) -> dict:
    """Deja solo los campos que declara el modelo."""
    return {c: v for c, v in datos.items() if c in modelo.model_fields}

def _feature_normalizado(datos: dict) -> dict:
    limpio = _recortar(UsgsFeature, datos)
    limpio["properties"] = _recortar(UsgsProperties, datos.get("properties") or {})
    geo = datos.get("geometry")
    limpio["geometry"] = _recortar(UsgsGeometry, geo) if geo else None
    return limpio
```

6.2. Cache TTL y resultado enriquecido

`get_latest_event` devuelve un `ExternalResult` con el sismo, si fue *hit* de cache y la latencia medida. `usar_cache=False` fuerza la llamada real (necesario para medir latencias).

Listing 10: Consulta con cache TTL

```python
@dataclass(frozen=True, slots=True)
class ExternalResult:
    feature: UsgsFeature | None
    cache_hit: bool
    latency_ms: float

async def get_latest_event(self, usar_cache: bool = True) -> ExternalResult:
    inicio = time.perf_counter()
    if usar_cache:
        guardado = await self._cache.get(self._feed_url)
        if guardado is not None:
            return ExternalResult(guardado, True,
                                  (time.perf_counter() - inicio) * 1000)
    sismo = await _encadenar_usgs(self._client, self._feed_url, self._timeout)
    if sismo is not None:
        await self._cache.set(self._feed_url, sismo)
    return ExternalResult(sismo, False, (time.perf_counter() - inicio) * 1000)
```

6.3. C8 - Medicion de latencia (media y p95)

Se calienta la cache y luego se mide: **con cache** (hits) y **sin cache** (llamadas reales). El percentil es por *rango mas cercano* (*nearest-rank*), simple de explicar.

Listing 11: Percentil y resumen de latencia

```python
def _percentil(valores: list[float], porcentaje: float) -> float:
    if not valores:
        return 0.0
    ordenados = sorted(valores)
    indice = ceil(porcentaje / 100 * len(ordenados)) - 1
    return ordenados[max(0, min(indice, len(ordenados) - 1))]

async def medir_latencia(self, muestras: int = 5):
    await self.get_latest_event()                # calienta la cache
    con = [(await self.get_latest_event()).latency_ms for _ in range(muestras)]
    sin = [(await self.get_latest_event(usar_cache=False)).latency_ms
           for _ in range(muestras)]
    return {"con_cache": _resumen(con), "sin_cache": _resumen(sin)}
```

7. Pruebas automatizadas (`tests/clients/`)

Se usan **pytest** + **pytest-asyncio** + **respx** (para simular HTTP sin red). `conftest.py` limpia el estado global (`_no_antes_de`, cache de modulo) y reemplaza `_dormir` para medir las esperas *sin dormir de verdad*.

Prueba	Que demuestra
`test_503_luego_200_reintenta`	Reintenta y termina en OK
`test_429_respeta_retry_after`	Usa `Retry-After`, no el backoff
`test_400_no_se_reintenta`	400 no se reintenta
`test_404_lectura_no_se_reintenta`	404 → `NoEncontrado`
`test_503_persistente_agota_reintentos`	Agota intentos → error
`test_timeout_persistente_...`	Timeout → `TimeoutDependencia`
`test_paginacion_con_fallo_...`	Reintenta por pagina
`test_respuesta_invalida_listado/lectura`	Pydantic → `RespuestaInvalida`
`test_degradacion_parcial_lecturas`	Un fallo no tumba al resto
`test_salud_y_sensor_existe`	`salud()` y existencia
`test_no_antes_de_compartido`	El bloqueo por IP es global
`test_lecturas_concurrentes_...`	Concurrencia respeta el bloqueo
`test_encadena_dos_llamadas_y_usa_cache`	Feed → detalle + cache
`test_sin_detalle_devuelve_...`	Evento sin `detail`
`test_feed_invalido_...`	Feed invalido → error
`test_timeout_lanza_timeoutdependencia`	Timeout de USGS
`test_obtener_ultimo_sismo_devuelve_hit`	Funcion de contrato
`test_percentil_y_resumen_latencia`	Matematica del p95
`test_medir_latencia_compara_...`	Con cache: 1 llamada real

Cuadro 4: Las 20 pruebas de `tests/clients/`.

Listing 12: Conftest: jitter y bloqueo medidos sin dormir

```python
@pytest.fixture
def dormidas(monkeypatch):
    registro: list[float] = []
    async def falso_dormir(segundos: float) -> None:
        registro.append(segundos)
    monkeypatch.setattr(us, "_dormir", falso_dormir)
    return registro
```

Resultado de correr toda la suite:

Listing 13: Salida real de pytest

```
$ ./venv/bin/python -m pytest tests/clients -q
....................                                                     [100%]
20 passed in 0.20s
```

8. Evidencias reales

Toda la evidencia se genera **contra el servicio real** (no simulada).

8.1. Reintentos, `Retry-After` y degradacion parcial

Fragmento de `evidencias/logs_clients.txt` (`-semilla 42 -tasa-fallos 0.4`). Se ve el backoff creciendo, el `Retry-After` ganando y el 404 que *no* se reintenta.

Listing 14: Evidencia real de reintentos (fragmento)

```
intento 1/5, causa=503, espera=0.170s, origen=backoff
intento 2/5, causa=503, espera=0.564s, origen=backoff
intento 3/5, causa=429, espera=2.000s, origen=Retry-After
...
GET /sensores/S-999/lectura "HTTP/1.1 404 Not Found"   <-- NO se reintenta

```

```
== obtener_lecturas (degradacion parcial) ==
  lecturas ok: 2  fallidos: 2
  ok:      {'sensor_id': 'S-001', 'tipo': 'temperatura', 'valor': 28.3, ...}
  fallido: {'sensor_id': 'S-003', 'motivo': '...sigue devolviendo 429...'}
  fallido: {'sensor_id': 'S-999', 'motivo': 'No existe el sensor S-999'}

== listar_sensores (recorre toda la paginacion) ==
total sensores: 23
```

8.2. Latencia con y sin cache (C8)

Corrida real contra USGS (`evidencias/latencia_cache.txt`).

Modo	Muestras	Media (ms)	p95 (ms)
✓ Con cache	5	0.002	0.003
× Sin cache	5	479.529	1174.669

Cuadro 5: Latencia de la API externa con y sin cache (5 muestras).

Lectura: la lectura de cache es sub-milisegundo mientras que la llamada real ronda los 480 ms de media (p95 $\approx$ 1.17 s). El TTL reduce latencia *y* carga sobre la dependencia externa.

9. Hallazgos para el equipo (analisis)

Estos son problemas que detecte al probar el sistema completo y que corresponden a otras partes:

Quien	Hallazgo	Recomendacion
Nataly	`Sensor` sin campo `tipo` y con `extra="forbid"`; el servicio devuelve `tipo`.	Agregar `tipo: str | None = None` o relajar a `extra="ignore"`.
Nataly	`UsgsFeed/UsgsProperties` con `extra="forbid"` vs. extras reales de USGS.	Cambiar a `extra="ignore"` (defensa en profundidad; yo ya lo tolero).
Nataly	Faltan constantes del contrato y `obtener_o_calcular` con *single-flight*.	Alinear `config.py`/`cache.py` al contrato acordado.
Daniela	`NoEncontrado` cae en el 502 en vez de 404.	Agregar `except NoEncontrado: HTTPException(404)`.
Daniela	Misma `Idempotency-Key` con cuerpo distinto devuelve la anterior (deberia ser 422).	Guardar un hash del cuerpo y comparar.
Daniela	Lock global en vez de por clave; `/docs` sin `summary`/`responses`.	Lock por clave y `responses={...}`.
Grupal	Faltan `requests.http`, tabla de latencias en README, demo de idempotencia y deps de test.	Completar entregables.

Cuadro 6: Hallazgos y recomendaciones (fuera de mi alcance).

Evidencia del 502/504 con la config "fuerte": con `-tasa-fallos 0.4` y el `max_retries=3` de `config.py`, las peticiones a veces agotan intentos antes que los 5 del contrato, y el endpoint responde 502/504. Es un sintoma de la falta de alineacion de constantes, no del cliente.

10. Conclusiones

Requisito	Estado	Donde
A1 Consumo de API externa real	✓	`external_api.py`
A2 Validacion Pydantic	✓	`external_api.py`
A3 Manejo de errores de la API	✓	`external_api.py`
C2 Timeout explicito	✓	`unstable_service.py`
C3 Reintentos selectivos	✓	`unstable_service.py`
C4 Backoff + jitter	✓	`unstable_service.py`
C5 `Retry-After` + bloqueo IP	✓	`unstable_service.py`
C6 Semaforo de concurrencia	✓	`unstable_service.py`
C7 Paginacion resiliente	✓	`unstable_service.py`
C8 Latencia media/p95 con y sin cache	✓	`external_api.py`
Pruebas automatizadas (20)	✓	`tests/clients/`
Evidencia real	✓	`evidencias/`

Cuadro 7: Estado final de mi parte.

Mi parte queda **completa**: consumo resiliente de las dos dependencias, cache con metricas de latencia, 20 pruebas en verde y evidencia real. Lo pendiente esta fuera de mi alcance y se detalla en la seccion anterior.

A. Guion sugerido para la exposicion

1. **Contexto (30 s):** "una API agregadora que depende de un servicio que falla y de una API publica; mi parte es sobrevivir a esos fallos".
2. **Arquitectura (30 s):** mostrar la figura 1.
3. **Excepciones (30 s):** figura 2; enfatizar la compatibilidad con el router.
4. **Resiliencia (2–3 min):** recorrer C2–C7 con un fragmento cada uno; mencionar *por que* el timeout de lectura es 2 s (menor que la lentitud simulada).
5. **API externa (1 min):** encadenado feed→detalle, y *contar el hallazgo* de `extra="forbid"` y como lo resolvi.
6. **Latencias (1 min):** tabla con/sin cache (0.002 ms vs 479 ms).
7. **Pruebas y evidencia (1 min):** 20 pruebas, log real.
8. **Hallazgos (1 min):** tabla de recomendaciones al equipo.

B. Comandos de reproduccion

Listing 15: Como reproducir todo

```
# 1) Servicio inestable (lo entrega el docente; no vive en el repo)
python servicio_inestable.py --semilla 42 --tasa-fallos 0.4

# 2) Pruebas de los clientes
./venv/bin/python -m pytest tests/clients -q

# 3) Evidencia real de reintentos
PYTHONPATH=. ./venv/bin/python tests/clients/evidencia_real.py

```

```
# 4) Medicion de latencia (con y sin cache)
PYTHONPATH=. ./venv/bin/python tests/clients/medir_latencia.py

# 5) Levantar la API y ver /docs
./venv/bin/python -m uvicorn main:app --host 127.0.0.1 --port 8000
```